MANUEL TECNICO MEDLEXER

INTRODUCCION:

EL siguiente manual tiene como objetivo principal transmitir las ideas para la realización de este proyecto, enfocándose en el análisis léxico para la obtención de tokens extraídos de un archivo.med

Analizador léxico:

La primera fase del analizador léxico esta en la creación de su header en donde establecemos la estructura que tendrá al momento de implementar los métodos dentro de su archivo de c++,

```cpp
// LexicalAnalyzer.h
#pragma once
#include "token.h"
#include <vector>
#include <string>
#include "Error.h"
class LexicalAnalyzer {
public:
    LexicalAnalyzer(const std::string& input);
    Token nextToken();
    std::vector<Token> getAllTokens();
    bool isEOF() const;
    const std::vector<LexicalError>& getErrors() const {
        return errorManager.getErrors();
    }
private:
    std::string source;
    size_t pos;
    int linea;
    int columna;
    ErrorManager errorManager;

    char peek() const;
    char advance();
    Token recognizeFecha();
    Token recognizeHora();
    Token recognizeCadena();
    Token recognizePalabra();
    Token recognizeEntero   ();
    Token recognizeCodigoID();
    Token recognizeTipoSangre();
    Token recognizeDosis();
```

Dentro del header se establecen los métodos públicos que se usaran en la clase, el mas importante por rendimiento es nextToken, ya que dentro de esta clase hará el llamado a las demas clases para establecer el tipo de token que se esta evaluando.

También definimos cada tipo de token que se estará evaluando el private, hora, fecha, código, diagnostico, dosis, clasificación que se estableció dentro de las instrucciones del proyecto.

```cpp
class LexicalAnalyzer {
public:
    LexicalAnalyzer(const std::string& input);
    Token nextToken();
    std::vector<Token> getAllTokens();
    bool isEOF() const;
    const std::vector<LexicalError>& getErrors() const {
        return errorManager.getErrors();
    }
private:
    std::string source;
    size_t pos;
    int linea;
    int columna;
    ErrorManager errorManager;

    char peek() const;
    char advance();
    Token recognizeFecha();
    Token recognizeHora();
    Token recognizeCadena();
    Token recognizePalabra();
    Token recognizeEntero   ();
    Token recognizeCodigoID();
    Token recognizeTipoSangre();
    Token recognizeDosis();

};
```

Una vez terminado este proceso, podemos empezar con el analizador léxico en su archivo de c++

```cpp
// LexicalAnalyzer.cpp
#include "AnalizadorLexico.h"
#include "Error.h"
#include <algorithm>

LexicalAnalyzer::LexicalAnalyzer(const std::string &input)
    : source(input), pos(0), linea(1), columna(1) {}

char LexicalAnalyzer::peek() const
{
    return pos < source.size() ? source[pos] : '\0';
}

char LexicalAnalyzer::advance()
{
    char c = peek();
    pos++;
    columna++;
    if (c == '\n')
    {
        linea++;
        columna = 1;
    }
    return c;
}

bool LexicalAnalyzer::isEOF() const
{
    return pos >= source.size();
}

Token LexicalAnalyzer::recognizeFecha()
```

En este archivo establecemos funciones de los métodos que agregamos en el header.

Peek():

Peek sirve para establecer el punto de inicio del analizador, la posición 0 del archivo que es su punto de partido, el comienzo de nuestro documento .med

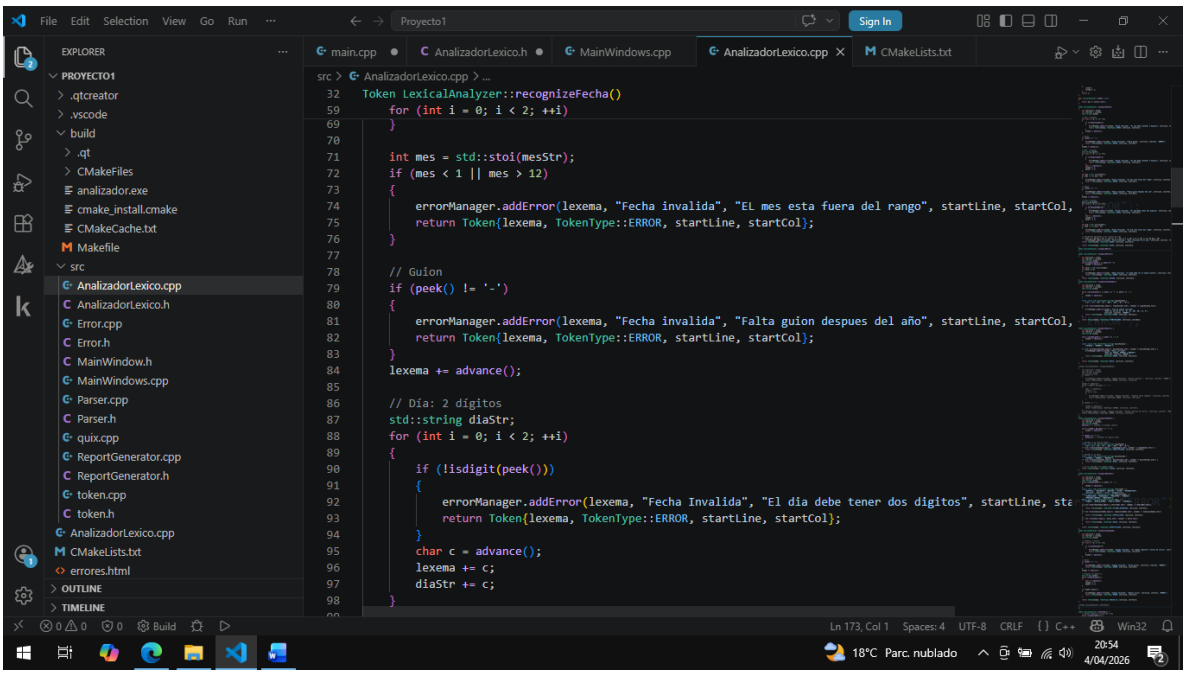
Advance():

Advance tiene como finalidad avanzar entre espacios en el documento hasta que se pueda reconocer un lexema, todo esto se agrega ala línea y columna, para tener en control la posición del analizador.

isEof(): establece la posición del analizador para poder ver cuantas líneas y columnas avanza el analizador.

recognizeFecha()

en este método el analizador toma los token fecha, analizando sus parámetros como la cantidad de dígitos en años, meses y días.



recognizeHora():

En este método token, guardamos los tokens que son reconocidos como horas, evaluando que se cumplan las restricciones en minuto y segundos, de cumplir todas el tipo de token queda como HORA.

```cpp
Token LexicalAnalyzer::recognizeFecha()
    if (dia < 1 || dia > 31)
    }

    // Validación adicional: días válidos por mes
    if ((mes == 2 && dia > 29) || (mes == 4 || mes == 6 || mes == 9 || mes == 11) && dia > 30)
        errorManager.addError(lexema, "Fecha Invalida", "Dias invalidos en el mes", startLine, startCol, "ERROR");
    return Token{lexema, TokenType::ERROR, startLine, startCol};

    return Token{lexema, TokenType::FECHA, startLine, startCol};
}
Token LexicalAnalyzer::recognizeHora()
{
    int startLine = linea;
    int startCol = columna;
    std::string lexema;

    // Hora: 2 dígitos
    std::string horaStr;
    for (int i = 0; i < 2; ++i)
    {
        if (!isdigit(peek()))
        {
            errorManager.addError(lexema, "Hora Invalida", "La hora necesita dos digitos", startLine, startCol, "ERROR");
            return Token{lexema, TokenType::ERROR, startLine, startCol};
        }
        char c = advance();
        lexema += c;
        horaStr += c;
    }

    // Dos puntos
```

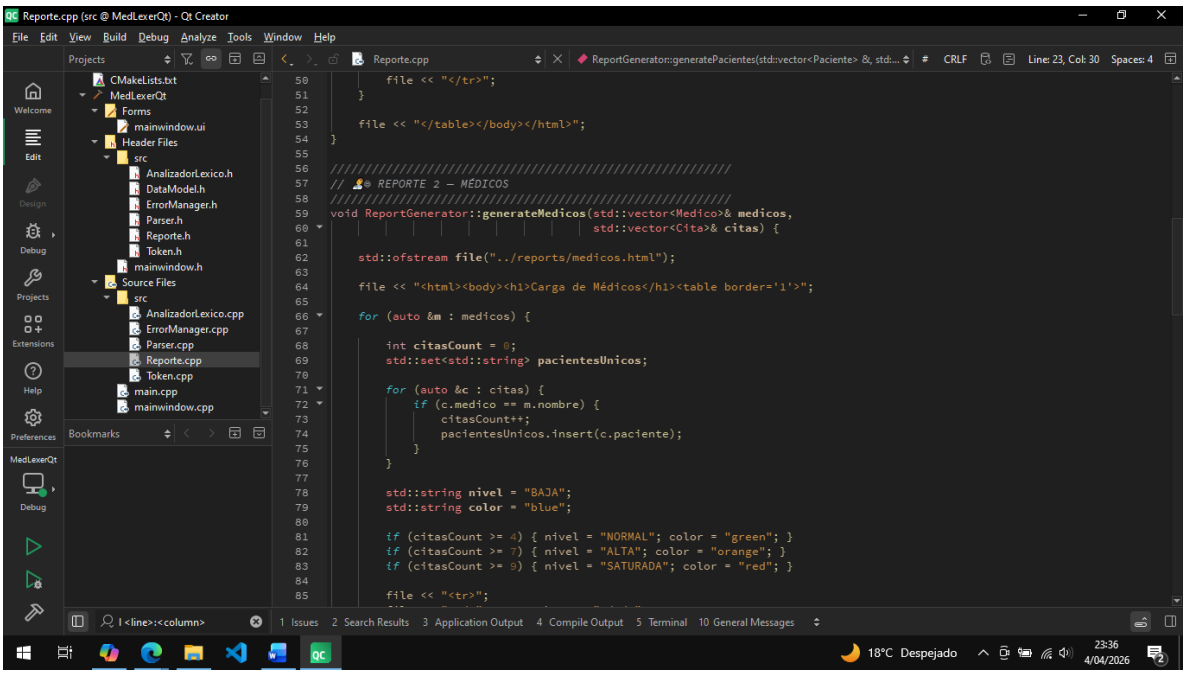
recognizeEntero()

en el método para reconocer entero, se toman restricciones que van desde el guion hasta el tipo de dato sea un dato int.

```cpp
Token LexicalAnalyzer::recognizeHora()
    if (minutos < 0 || minutos > 59)
        errorManager.addError(lexema, "Hora Invalida", "Minutos fuera de rango", startLine, startCol, "ERROR");
        return Token{lexema, TokenType::ERROR, startLine, startCol};
    }

    return Token{lexema, TokenType::HORA, startLine, startCol};
}

Token LexicalAnalyzer::recognizeEntero()
{
    int startLine = linea;
    int startCol = columna;
    std::string lexema;
    while(isdigit(peek()) || peek()=='-'){
        lexema += advance();
    }
    int valor = std::stoi(lexema);
    if (valor <= 0)
    {
        errorManager.addError(lexema, "Edad Invalida", "La edad debe ser un numero entero", startLine, startCol, "ERROR");
        return Token{lexema, TokenType::ERROR, startLine, startCol};
    }
    return Token{lexema, TokenType::ENTERO, startLine, startCol};
}
Token LexicalAnalyzer::recognizeTipoSangre()
{
    int startLine = linea;
    int startCol = columna;
    std::string lexema;

    while (isalnum(peek()) || peek() == '+' || peek() == '-')
```

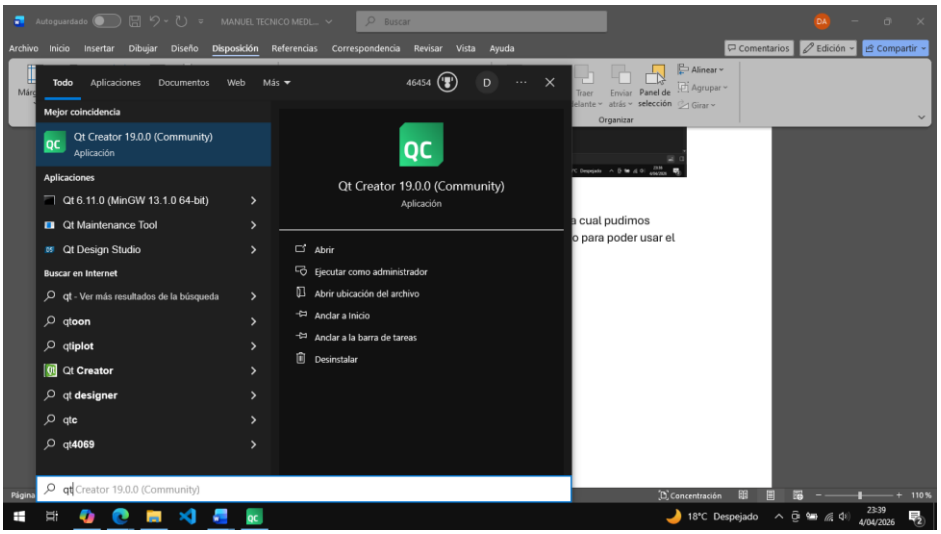
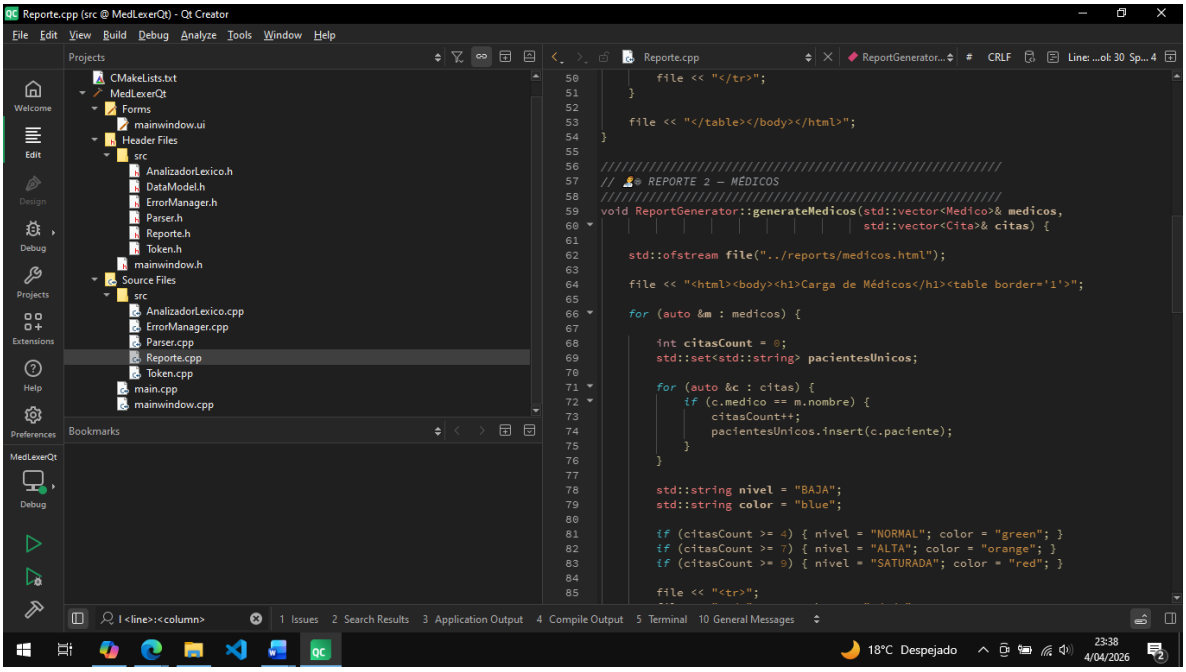
Generador de reportes:

La generación de rpotes se hizo por medio de métodos dentro de la clase generación de reportes, en este método se crea la estructura de un html que al reconocer cada objeto que sea ingresado dentro de el se convierte en un archivo html que el usuario puede visualizar en una carpeta.



Interfaz grafica:

La interfaz grafica se hizo por medio de la aplicación de Qt con la cual pudimos realizar diferentes arreglos para el formulario que tiene el usuario para poder usar el programa.



Los modelos usados para este proyecto son la estructura del mismo, los archivos de c++ y los headers, y los mainwindow que se usan en qt para crear la interfaz.